

Starbot — Developer Documentation

Project: Starbot (RAG-OUTLOOK) — an enterprise AI assistant for AppXcess Technologies that lets employees query their Microsoft 365 data (Outlook, SharePoint, OneDrive) in plain English, auto-classifies their inbox, and generates documents — all grounded in real company data with citations.

Document scope: Current implementation, developer's view. | **Date:** 2026-06-09

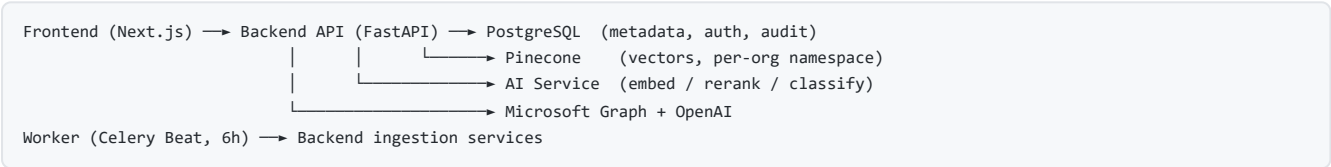
1. Architecture Overview

The system is a multi-service application split into four runtime components:

Component	Tech	Port	Responsibility
Backend API	Python, FastAPI, SQLAlchemy (async)	3001	Auth, search/RAG, mail, documents, ingestion, chat — all REST under <code>/api/v1</code>
AI Service	Python, FastAPI, Sentence-Transformers	8001	Stateless microservice: embeddings, reranking, intent classification
Worker	Celery + Celery Beat	—	Scheduled background ingestion (every 6h)
Frontend	Next.js 15, React 19, Tailwind, Radix UI	3000	Web UI (App Router), proxies <code>/api/v1/*</code> to backend

Backing services: PostgreSQL (metadata, permissions, audit), Pinecone (vector store, 384-dim), Redis (cache + OAuth state, optional with in-memory fallback).

External APIs: Microsoft Graph (mail/files/sites), OpenAI (`gpt-5.4-mini`), Tavily (business web search, optional).



Repo layout: `backend/app` (api: `router/`, `services/`, `models/`, `schema/`, `prompts/`, `core/`, `job/`), `backend/ai-service`, `frontend/`, `infrastructure/` (postgres migrations, docker, k8s, terraform).

2. Authentication

Microsoft Entra OAuth 2.0 with JWT session cookies.

- `GET /auth/microsoft/authorize-url` → generates a `state` (stored in Redis, 10-min TTL) and returns the Entra consent URL.
- User consents → `GET /auth/microsoft/callback` validates `state`, exchanges the code for tokens.
- Profile fetched from Graph `/me`; user upserted; **OAuth tokens encrypted with AES-256-GCM** and stored in `oauth_tokens`.
- A JWT (HS256) is issued as an httpOnly cookie (`starbot_session`); a row is stored in `sessions`.
- Protected routes use the `CurrentUserDep` dependency to validate the JWT and load the user context.

Scopes: `User.Read`, `Mail.Read`, `Mail.ReadWrite`, `Files.Read.All`, `Sites.Read.All`, `offline_access`. **Roles:** `owner` / `admin` / `member` / `viewer` (in `organization_members`). All data is scoped by `organization_id` + `user_id` at the application layer.

3. Data Ingestion Pipeline

Pulls M365 content into the searchable index. (`services/ingestion_service.py`, `job/tasks/ingestion.py`)

- Trigger:** Celery Beat `scheduler.tick` runs every 6h (`INGESTION_SYNC_INTERVAL_MS`), enqueueing one sync task per user per source. Manual triggers: `POST /ingestion/outlook/sync`, `POST /ingestion/documents/sync`.
- Outlook:** fetches ~100 recent emails via Graph → stores metadata in `email_metadata` → chunks `subject + body`.
- SharePoint/OneDrive:** crawls files (OneDrive uses **delta-sync cursors**; SharePoint full crawl) → stores metadata in `file_metadata` → extracts text from `.docx/.pdf/.xlsx/.pptx` (python-docx, pypdf, openpyxl); metadata-only fallback if extraction fails.
- Common path:** `chunk_text(size=512, overlap=64)` → embed via AI service → **upsert to Pinecone** in batches of 100, namespace = `organization_id`, vector id = `{source}:{contentId}:{chunkId}`, with `source` metadata.

4. Search & RAG Flow

How a question becomes a cited answer. (`services/rag_service.py`, `query_service.py`, `hybrid_retrieval`)

- Topic guard** — off-topic queries (weather, sports, etc.) are refused before any LLM call.
- Intent routing** — AI service `/agent/classify` returns intent + suggested sources. Recency phrasing ("latest emails") narrows to Outlook; ("recent files") to SharePoint/OneDrive.
- Hybrid retrieval (parallel):**
- Keyword** — SQL `ILIKE` on email subjects / file names (limit 5).
- Vector** — embed query (384-dim) → Pinecone `query(top_k=20)` filtered by `source`, `organizationId`, and optional `workspaceId` / `projectId`.

6. **Permission filter** — drops chunks the user doesn't own.
7. **Fusion + rerank** — Reciprocal Rank Fusion (K=60) merges keyword+vector, then AI service `/rerank` returns top 5.
8. **Live fallback** — if vectors are empty, fetch fresh emails/files directly from Graph.
9. **Synthesis** — OpenAI `gpt-5.4-mini` (temp 0.2) with `RAG_SYSTEM_PROMPT` + ranked context blocks → answer **with citations** (source, title, snippet, URL). Streamed to client via SSE; results cached in Redis (120s TTL, skipped for live data).

Endpoints: `POST /search/unified` , `POST /chat/sessions/{id}/messages/stream` .

5. Mail Organization (Smart Mail Agent)

LLM-based inbox triage and reply drafting. (`services/mail_service.py` , `router/mail.py`)

- `POST /mail/classify` — fetches recent inbox emails via Graph, classifies each into **important / spam / closed / pending_action** using OpenAI; results saved to `email_classifications` with confidence + reasoning.
- `GET /mail/classifications` — returns stored classifications; `POST /mail/bulk-dismiss` soft-deletes spam.
- `POST /mail/reply/generate` — generates AI reply drafts for selected emails.
- `POST /mail/reply/draft` — saves the reviewed reply back into Outlook as a draft via Graph.

6. Document Generation

Template-driven, optionally grounded in M365 data. (`services/documents_service.py` , `router/documents.py`)

1. Request detected via `/document` prefix or template keywords; `POST /documents/generate` .
2. LLM extracts the **template type** (`estimate / job_summary / report / quotation / customer_email`) and variables from the user's description.
3. If the request references M365 sources (regex on "sharepoint/onedrive/email/invoice/report..."), relevant context is retrieved via the RAG pipeline first.
4. **Two paths:** *template-based* (fill `{variable}` placeholders from a `document_templates` row, then LLM-polish) or *free-form grounded* (generate from retrieved M365 content when no template matches).
5. Response returns a draft (body + optional subject, `canSaveToOutlook` flag). `POST /documents/export` renders to PDF/DOCX; `POST /documents/draft-email` saves an email draft.

Templates are CRUD-managed via `GET/POST /documents/templates` , `PATCH/DELETE /documents/templates/{id}` .

7. Other Features

- **Workspaces** — department contexts (Sales, Operations...) with versioned `workspace_instructions` injected into RAG prompts. (`GET /workspaces` , `PUT /workspaces/{id}/instructions`)
- **Projects** — ChatGPT-style per-user knowledge containers with uploaded files + custom instructions; files are chunked/embedded into Pinecone (`source=project`) and used for project-scoped retrieval, organized into sectors. (`/projects` CRUD + `/files` + `/sectors`)
- **Audit logs** — every sensitive action recorded in `audit_logs` ; admin/owner read via `GET /audit/logs` .

8. AI Service (`backend/ai-service`)

Standalone FastAPI microservice. (`ai-service/app/main.py`)

Endpoint	Function
<code>POST /embed</code>	Sentence-Transformers <code>all-MiniLM-L6-v2</code> , 384-dim , normalized
<code>POST /agent/classify</code>	Lightweight regex intent router → <code>mail / documents / workspace / general</code> + suggested sources
<code>POST /rerank</code>	Token-overlap rescoring (<code>base*0.6 + overlap*0.4</code>), returns top-K

9. Data Model (PostgreSQL — key tables)

`users` , `oauth_tokens` , `sessions` , `organizations` , `organization_members` (auth) · `workspaces` , `workspace_instructions` · `chat_sessions` , `chat_messages` (citations + metadata as JSONB) · `email_metadata` , `email_classifications` · `file_metadata` (graph_item_id, source, drive_id, site_id) · `document_templates` · `projects` , `project_files` , `project_sectors` · `sync_cursors` (delta-sync state) · `ingestion_jobs` · `audit_logs` .

Note: Vectors live only in Pinecone; PostgreSQL holds searchable metadata, permissions, and audit. Multi-tenant isolation is per-org (Pinecone namespace + SQL `organization_id` scoping).

10. Configuration & Run

Key env vars: `DATABASE_*` , `JWT_SECRET` , `AZURE_CLIENT_ID/SECRET/TENANT_ID` , `TOKEN_ENCRYPTION_KEY` (AES-256-GCM), `OPENAI_API_KEY` , `PINECONE_API_KEY` , `PINECONE_INDEX_NAME` , `AI_SERVICE_URL` , `REDIS_URL` , `WORKER_SERVICE_TOKEN` , `INGESTION_SYNC_INTERVAL_MS` , `TAVILY_API_KEY` (optional).

Constants: embeddings 384-dim · `DEFAULT_TOP_K=20` · `RERANK_TOP_K=5` · chunk 512/overlap 64 · `LLM_MODEL=gpt-5.4-mini` .

Run: Docker Compose (`backend/docker-compose.yml`) brings up api, ai-service, worker, web, redis. Kubernetes manifests + Terraform in `infrastructure/` . DB schema via migrations in `infrastructure/postgres/migrations/` (Alembic also present in `backend/alembic`).